Green Computing in Healthcare

Binary Classification on Electronic Health Records
Energy Efficiency Analysis Across Three Implementations

1. Introduction and Motivation

The growing adoption of machine learning in clinical decision support raises important questions about computational sustainability. Training and evaluating models on Electronic Health Records (EHR) consumes energy and emits CO_2-equivalent greenhouse gases. This project benchmarks three implementations of a binary classification pipeline: a baseline in Python (Script 1), a green-optimised variant also in Python (Script 2), and an equivalent implementation in R (Script 3), across five real-world EHR datasets. Energy consumption is measured with CodeCarbon for the Python scripts and with wall-clock proxies for R. Classification performance is quantified by the Matthews Correlation Coefficient (MCC), a balanced metric well suited to class-imbalanced medical data.

2. Datasets

Five publicly available EHR datasets were used:

ID	Dataset	N	Features	Target	Pos. class
D1	Neuroblastoma (PeerJ 2018)	169	12	Outcome	45.6%
D2	Pediatric Brain Tumor (Belgrade 2021)	173	29	Status OS	54.3%
D3	Colorectal Cancer EHR (Taipei 2018)	999	27	Death	42.7%
D4	Sepsis / SIRS ICU	1,257	14	Mortality	10.3%
D5	Depression and Heart Failure	425	11	Death	23.3%

Table 1. Dataset summary. Pos. class = proportion of positive class.

Pre-processing was identical across all scripts: whitespace was stripped from column names, missing values were imputed with the column median, non-binary targets were remapped to {0, 1}, and all features were cast to float32. No handcrafted feature engineering was applied.

3. Methods

3.1 Evaluation Protocol

Repeated hold-out validation with 100 iterations was applied to every dataset and script. In each iteration a fresh 70/30 stratified split was drawn with seed equal to 42 plus the iteration index. The MCC was computed on the test fold and the mean and standard deviation were reported across the 100 repetitions. MCC was chosen over accuracy or AUC because it is invariant to class imbalance and summarises all four confusion-matrix cells into a single scalar.

3.2 Script 1: Baseline in Python

A Random Forest with 100 trees was trained in each of the 100 hold-out splits using scikit-learn with all available CPU threads. StandardScaler was re-fitted inside every iteration on the training fold only, preventing data leakage. No feature selection was applied. Energy consumption was measured with CodeCarbon in offline mode.

3.3 Script 2: Green Optimised in Python

Four energy-saving strategies were applied relative to Script 1:

- **Feature selection via SelectKBest with k=10:** applied once on the full dataset using ANOVA F-scores, reducing input dimensionality before any training occurs.
- **Scaler pre-fitted once:** StandardScaler is fitted on the full feature matrix at the start and reused across all splits, eliminating 100 redundant fit operations.
- **Reduced ensemble size:** 10 trees instead of 100, sufficient given the smaller pre-selected feature space.
- **Single-threaded execution with bounded depth:** n_jobs=1 avoids multiprocessing overhead on small datasets; max_depth=8 caps per-tree memory usage.

3.4 Script 3: Green Implementation in R

Script 3 mirrors the green strategy using R with the ranger package, a C++-backed Random Forest implementation that is considerably faster than base R randomForest. Feature selection relies on ANOVA F-scores computed via aov(). A custom MCC function avoids extra dependencies, and column names are sanitised before passing data to ranger. Because no CodeCarbon equivalent exists natively in R, energy is estimated from wall-clock time and a CPU TDP proxy.

4. Results

4.1 Classification Performance

Dataset	Script 1 MCC ± std	Script 2 MCC ± std	Script 3 (R) MCC ± std
Neuroblastoma	0.477 ± 0.102	0.447 ± 0.118	0.481 ± 0.113
Pediatric Brain Tumor	0.422 ± 0.101	0.380 ± 0.117	0.435 ± 0.120
Colorectal Cancer	0.130 ± 0.045	0.135 ± 0.049	0.149 ± 0.050
Sepsis / SIRS	0.519 ± 0.062	0.517 ± 0.065	0.518 ± 0.062
Depression and HF	0.327 ± 0.081	0.294 ± 0.087	0.313 ± 0.086

Table 2. Mean MCC ± standard deviation across 100 hold-out iterations.

MCC values are consistent across all three scripts. The largest drops between the baseline and the green Python variant occur on Neuroblastoma and Depression with Heart Failure, both small datasets where reducing the number of trees introduces more variance. On Sepsis, the most imbalanced dataset at 10% positive class, Script 2 nearly matches Script 1 despite using ten times fewer trees, which validates the feature-selection strategy. Colorectal Cancer shows uniformly low MCC across all scripts, consistent with weak class separability in this cohort.

4.2 Energy and Time Consumption

Metric	Script 1 Baseline, Py	Script 2 Green, Py	Script 3 Green, R	Reduction 1 → 2
Wall-clock time (s)	68.50	7.91	3.38	−88.5%
Energy (Wh)	0.2613	0.0335	~0.0047 *	−87.2%
CO2-eq (g)	0.001029	0.000132	~0.00002 *	−87.2%

The green Python script is 8.7 times faster and consumes 87% less energy than the baseline, at a mean MCC cost of only 0.015 across all datasets. The R script is the fastest overall at 3.38 seconds though its energy figures rely on CPU-TDP proxies rather than hardware counters. To contextualise the absolute values: even the baseline emits roughly 0.001 g CO2-eq, about 200 times less than a single Google search at approximately 0.2 g, which confirms that the relative comparison of 87% reduction is the meaningful result here.

5. Discussion

5.1 Effectiveness of Green Strategies

The dominant saving comes from the combination of feature selection applied once and the reduced tree count. On datasets with many features such as Pediatric Brain Tumor with 29 features and Colorectal Cancer with 27, SelectKBest reduces the decision-surface complexity before any tree is grown, compounding savings across all 100 splits. Pre-fitting the scaler outside the loop contributes a smaller but measurable saving on larger datasets such as Sepsis with 1,257 samples.

Single-threaded execution may seem counter-intuitive but avoids Python's multiprocessing fork overhead, which dominates runtime on datasets with fewer than roughly 2,000 samples. This aligns with the Green Computing principle of matching parallelism to actual workload size.

5.2 MCC as a Green-Aware Metric

MCC is appropriate for this project for two reasons. First, it handles class imbalance gracefully, which is critical for Sepsis at 10% positive and Depression with Heart Failure at 23%. Second, it provides a single number for comparing model quality across scripts, making the accuracy-versus-energy trade-off easy to quantify: the green script sacrifices on average 0.015 MCC units to achieve 87% energy reduction.

5.3 R versus Python

The R implementation outperforms both Python scripts in execution time. The ranger backend is especially efficient for small datasets. The main limitation is the absence of a native CodeCarbon equivalent in R; future work could wrap the script in a Python subprocess to measure hardware counters end-to-end. MCC values are closely aligned between R and Python scripts, within 1 to 2%, confirming that the algorithmic logic is equivalent across languages.

5.4 Limitations

Energy measurements from CodeCarbon in a containerised environment may underestimate real hardware consumption because CodeCarbon falls back to CPU-load estimation when hardware power interfaces are unavailable. The reported Wh figures should therefore be interpreted as relative.

6. Conclusions

This project demonstrates that significant energy savings are achievable in EHR classification without major sacrifice in predictive performance. Applying four lightweight optimisations reduced wall-clock time by 88.5% and energy consumption by 87.2% compared to the baseline Random Forest, while losing only 0.015 mean MCC units. The R implementation confirms that language choice can further amplify gains when a native compiled backend is available.

The results support the broader Green Computing lesson that algorithmic efficiency, choosing what not to compute, often outweighs hardware-level tuning for workloads at the scale typical of clinical EHR studies.

CodeBerg repository

https://codeberg.org/FilippoAvon